Assign 4

과제 풀이 정리 · Notion 아카이브

Problem 3 : *ModelBasedMonteCarlo*

3a - Value Iteration

```
def valueIteration(
succAndRewardProb:
Dict[Tuple[StateT, ActionT], List[Tuple[StateT, float, float]]],
discount: float, epsilon: float = 0.001
):
-----------------------------------------------------------------
stateActions = defaultdict(set)
for state, action in succAndRewardProb.keys():
stateActions[state].add(action)

def computeQ(V:Dict[stateT, float], state:stateT, action:ActionT) -> float:
ret = 0
for next_state, prob, reward in succAndRewardprob[(state,action)]:
ret += prob * (reward + discount * V[next_state])
return ret
-----------------------------------------------------------------
def computePolicy(V: Dict[stateT, float]) -> Dict[stateT,ActionT]
policy = {}
for state in stateActions:
policy[state] = max(stateActions[state],key=lambda a: (computeQ(V,state,a),a))
return policy
-----------------------------------------------------------------
print('Running valueIteration...')
 V = defaultdict(float)
 while True:
 newV = defaultdict(float)
 for state in stateActions:
 newV[state] = max(computeQ(V,state,action) for action in stateActions[state])
if max(abs(V[state]-newV[state]) for state in stateActions) <= epsilon:
break
V = newV
```

```
numIters += 1

V_opt = V
  print(("valueIteration: %d iterations" % numIters))
return computePolicy(V_opt)
```

3b÷ Model-Based Monte Carlo - 진짜 모르겠다

```
class ModelBasedMonteCarlo(util.RLAlgorithm):
def __init__(self, actions:List[ActionT],discount:float, calcValIterEvery: int = 10000,
explorationProb: float = 0.2,) -> None:
self.actions = actions
    self.discount = discount
    self.calcValIterEvery = calcValIterEvery
    self.explorationProb = explorationProb
    self.numIters = 0

    # (state, action) -> {nextState -> ct} for all nextState
    self.tCounts = defaultdict(lambda: defaultdict(int))
    # (state, action) -> {nextState -> totalReward} for all nextState
    self.rTotal = defaultdict(lambda: defaultdict(float))

    self.pi = {} # Optimal policy for each state. state -> action
----------------------------------------------------------------
def getAction(self, state: stateT, explore: bool = True) -> ActionT:
if expolre:
self.numIters += 1
  explorationProb = self.explorationProb
  if self.numIters < 2e4
  expolrationProb = 1.0
elif self.numIters > 1e6
expolationProb = expolationProb / math.log(self.numIters - 100000 + 1)

  if (expore and random.random()<explorationProb) or (state not in self.pi)
  return random.choice(self.actions)
else:
return self.pi[state]
----------------------------------------------------------------
def incorporateFeedback(self, state: StateT, action: ActionT, reward:int, nextState: stateT, terminal: bool)
```

```
  self.tCounts[(state, action)][nextState] += 1
  self.rTotal[(state, action)][nextState] += reward
  //-> state, action 쌍 => count +=1, reward +=reward => 기록

if self.numIters % self.calcValIterEvery == 0:
succAndRewardProb = defaultdict(list)
for (s,a), next_states in self.tCounts.items():
total_trans = sum(next_states.values())
for un, count in next_states.items():
prob = count / total_trans
reward = self.rTotal[(s,a)][ns] / count
succAndRewardprob[(s,a)].append((ns,prob,reward))
self.pi = valueIteration(succAndRewardProb, slef.discount)
```

Problem 4 : *Model-free RL*

4a : TabularQ-Learning(큐러닝)

```
class TabularQLearning(util.RLAlgorithm):
def __init__(self, actions: List[ActionT], discount: float,
                explorationProb: float = 0.2, initialQ: float = 0):
  self.actions = actions
    self.discount = discount
    self.explorationProb = explorationProb
    self.Q = defaultdict(lambda: initialQ)
    self.numIters = 0
---------------------------------------------------------------
def getAction(self, state: StateT, explore:bool=True) -> ActionT
if explore:
self.numIters += 1
explorationProb = self.explorationProb
if self.numIters < 2e4:
explorationProb = 1.0
elif self.numiters > 1e5:
explorationProb = explorationProb / math.log(self.numIters - 100000 + 1)

if (explore and random.random() < explorationProb)
return random.choice(self.actions)
else:
max_q = max(self.Q[state, a] for a in self.acitons)
```

best_actions = [a for a in self.actions if self.Q[state, a] == max_q]

return random.choice(best_actions)

---

def getStepSize(self) -> float:

return 0.1

---

def incorporateFeedback(self, state: StateT, action: ActionT, reward: float, nextState: StateT, terminal: bool) -> None:

a = self.getStepSize()

if terminal:

target_V = reward

else:

target_V = reward + self.discount * max([self.Q[nextState, act] for act in self.actions])]

self.Q[state,action] = (1-a)*self.Q[state,action] + a * target_V

4b÷ FourierFeature(특징)extractor

---

def fourierFeatureExtractor(

state: StateT,

maxCoeff: int = 5, //-> weight의 최댓값

scale: Optional[Iterable] = None

) -> np.ndarray:

if scale is None:

scale = np.ones_like(state)

features = None

---

scaled_state = np.array(state) * np.array(scale)

coeff_list = np.arrange(maxCoeff + 1) //-> [0,1,2,...,maxCoeff]

for s_s in scaled_state:

multiples = s_s * coeff_list

if feaures is None:

features = multiples

else:

features = np.add.outer(features, multiples).flatten()

features = np.cos(np.pi * features)

```
return features
```

4c÷Q-learning with Function Approximation

```
class FunctionApproxQLearning(util.RLAlgorithm):
def __init__(self, featureDim: int, featureExtractor: Callable, actions: List[int],
discount: float, explorationProb=0.2):

self.featureDim = featureDim
    self.featureExtractor = featureExtractor
    self.actions = actions
    self.discount = discount
    self.explorationProb = explorationProb
    self.W = np.random.standard_normal(size=(featureDim, len(actions)))
    self.numIters = 0
----------------------------------------------------------------
def getQ(self, state: np.ndarray, action: int) -> float:
features = self.featureExtractor(state)
weight = self.W[:,action]
return np.dot(weight, features)
----------------------------------------------------------------
def getAction(self, state: np.ndarray, explore: bool= true) -> int:
if explore:
self.numIters += 1
explorationProb = self.explorationProb
if self.numIters < 2e4
explorationProb = 1.0
elif self.numIters > 1e5
explorationProb = explorationProb / math.log(self.numIters - 100000 + 1)

if (explore and random.random() < explorationProb):
return random.choice(self.actions)
else:
max_q = max(self.getQ(state,a) for a in self.actions)
best_actions = [a for a in self.actions if self.getQ(state,a) == max_q]
return random.choice(best_actions)
----------------------------------------------------------------
def getStepSize(self) -> float:
```

```
return 0.005 * (0.99)**(self.numIters / 500)
----------------------------------------------------------------
def incorporateFeedback(self, state:np.ndarray,action:int,reward:float,nextState:np.ndarray,terminal:bool)->None:
a = self.getStepSize()
if terminal:
V_opt = 0.0
else:
V_opt = max([self.getQ(nextState, act) for act in self.actions])
features = self.featureExtractor(state)

expect = self.getQ(state, action)
features = self.featureExtractor(state)
self.W[:,action] = self.W[:,action] + a*(target - expect) * features
```

← 인공지능 · 포스텍 목차로